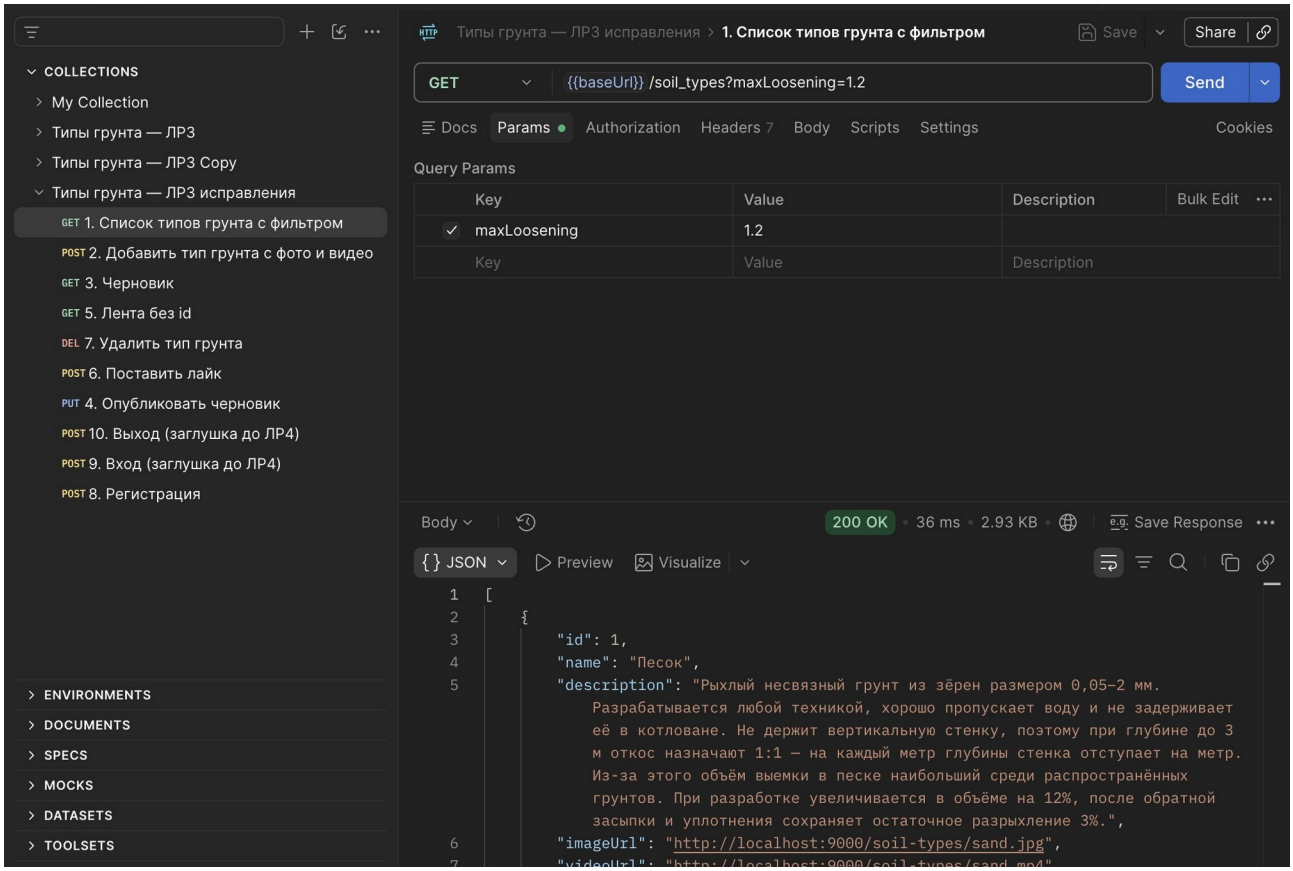
Лабораторная работа 3 Веб сервис

Давыдов Кирилл Игоревич, ИУ5-54Б

Тема — определение объёма земляных работ при разработке котлована

NestJS, TypeORM, PostgreSQL и MinIO. API содержит десять методов в двух доменах. Текущий пользователь в ЛР3 зафиксирован функцией getCurrentUserId, вход и выход остаются заглушками. Скриншоты выполнены на отдельной базе с согласованными данными.



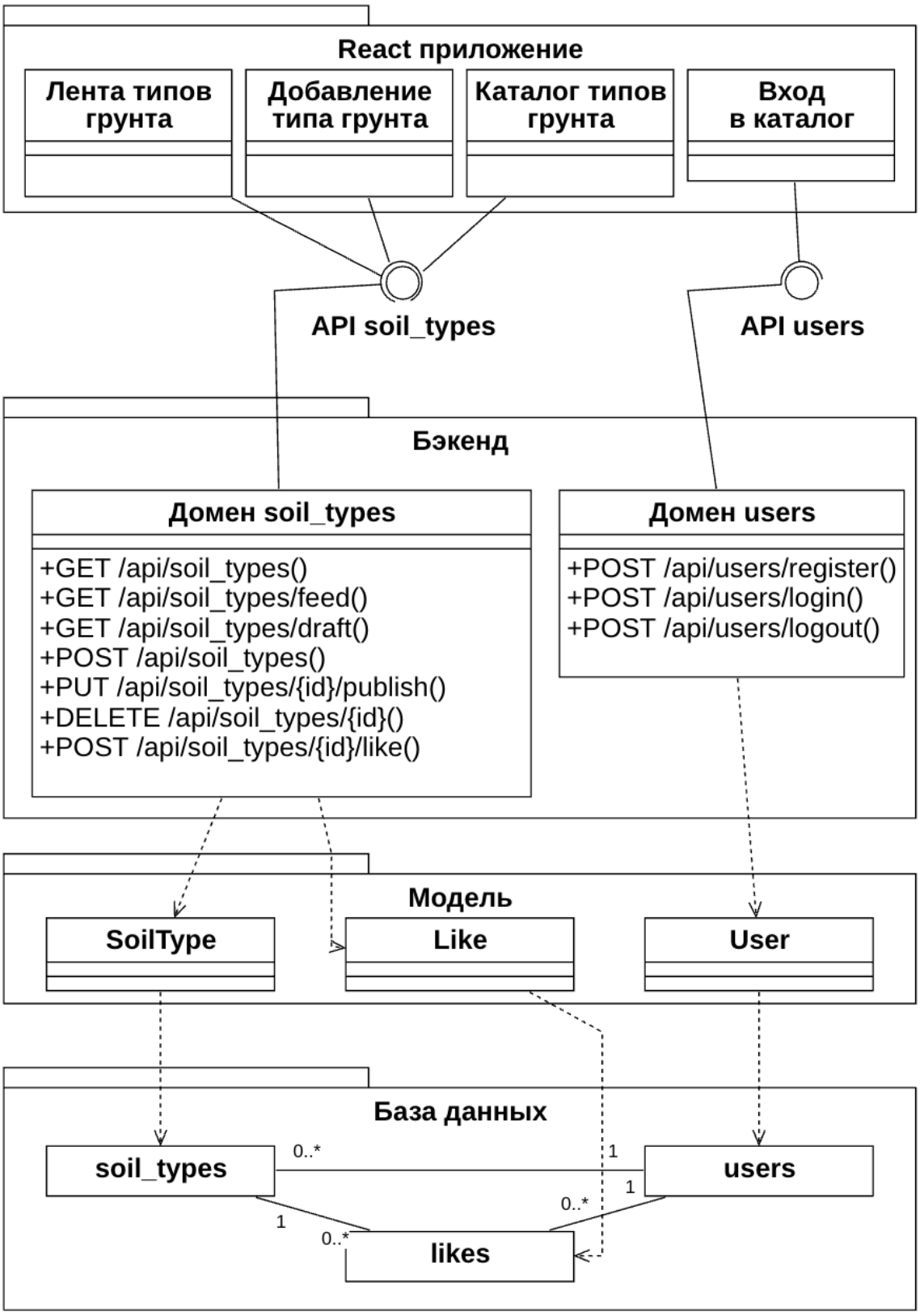
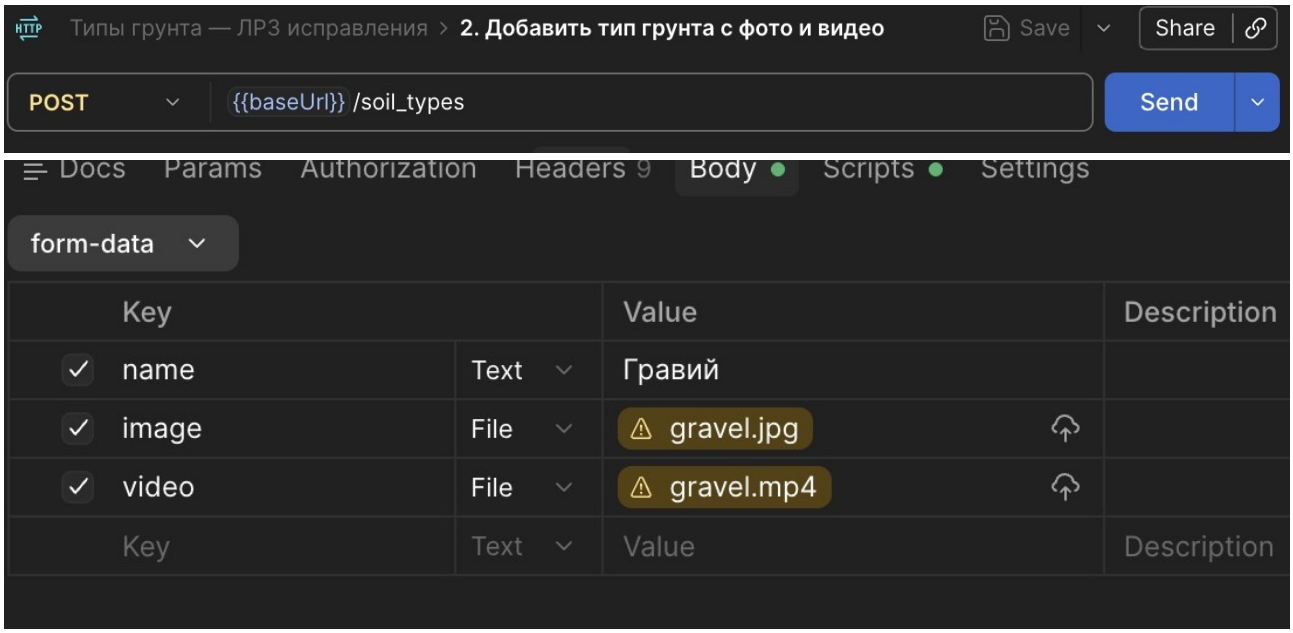
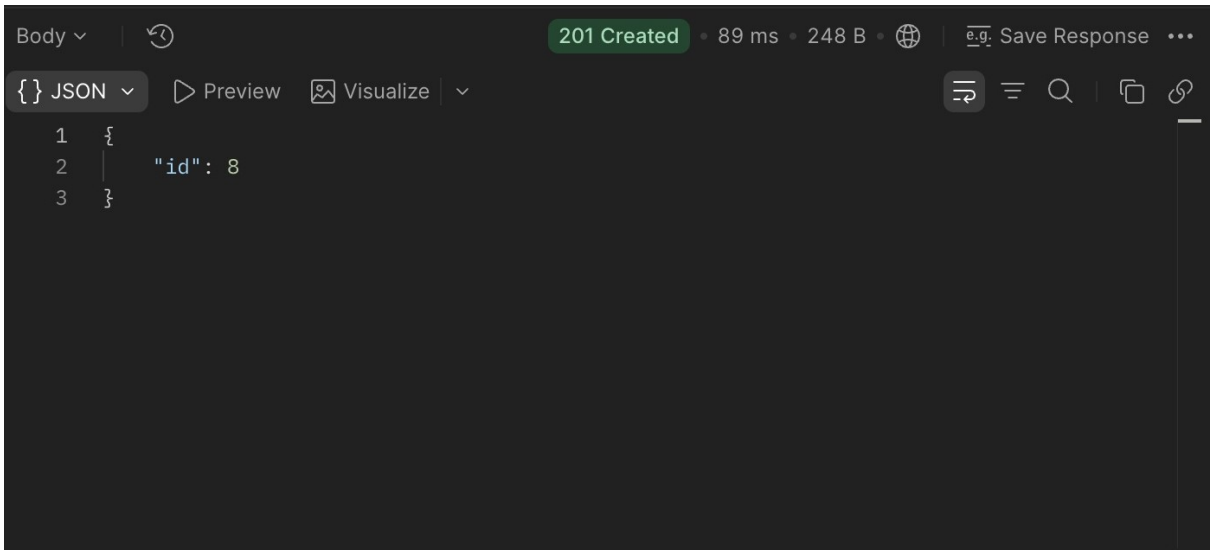


Диаграмма классов из общего файла soil_types.mdj

Скриншот 2 Добавление с фото и видео

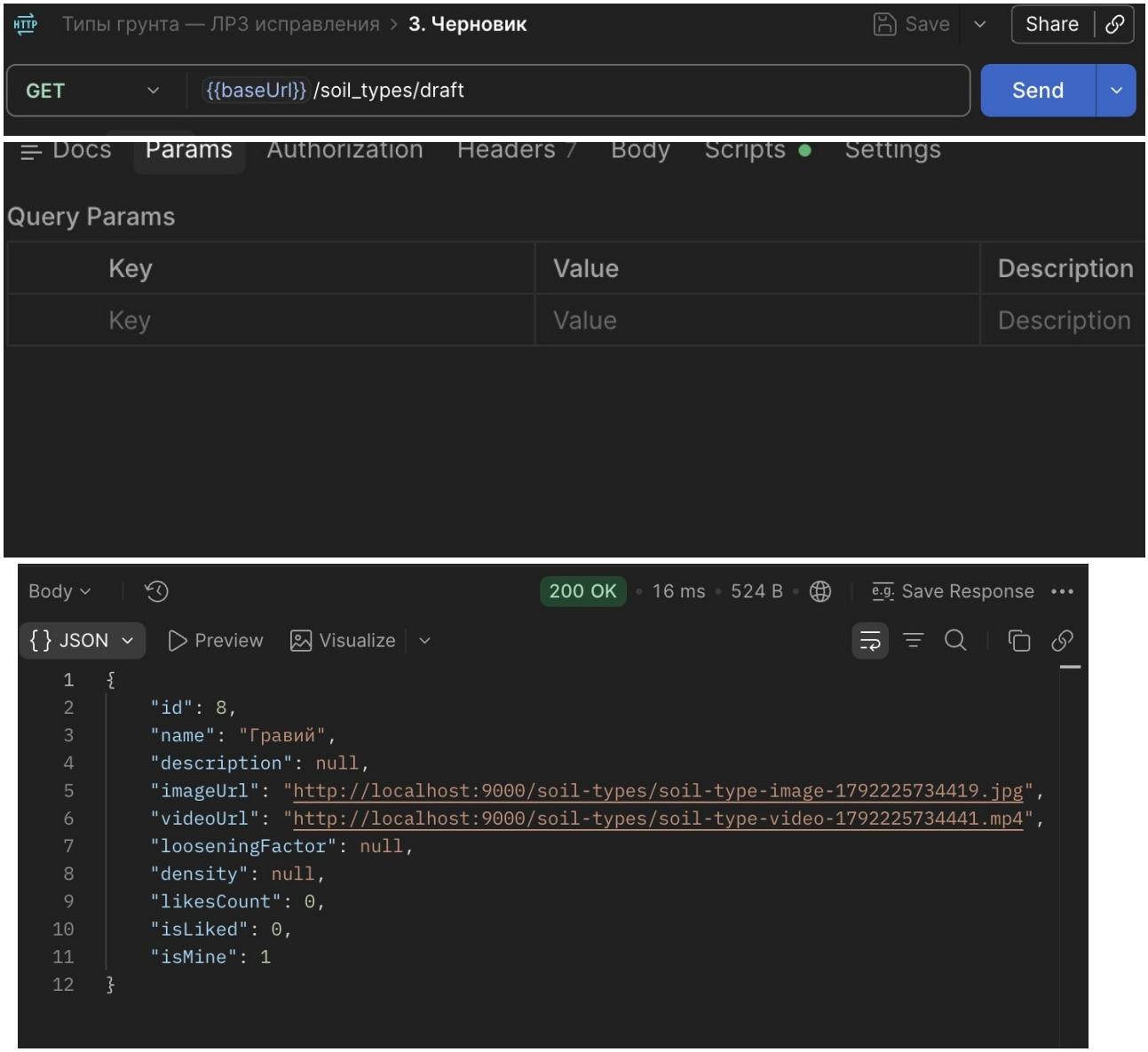
POST /api/soil_types. Вход: name, image, video. Ответ 201 содержит только id. Перед созданием черновик глины опубликован.





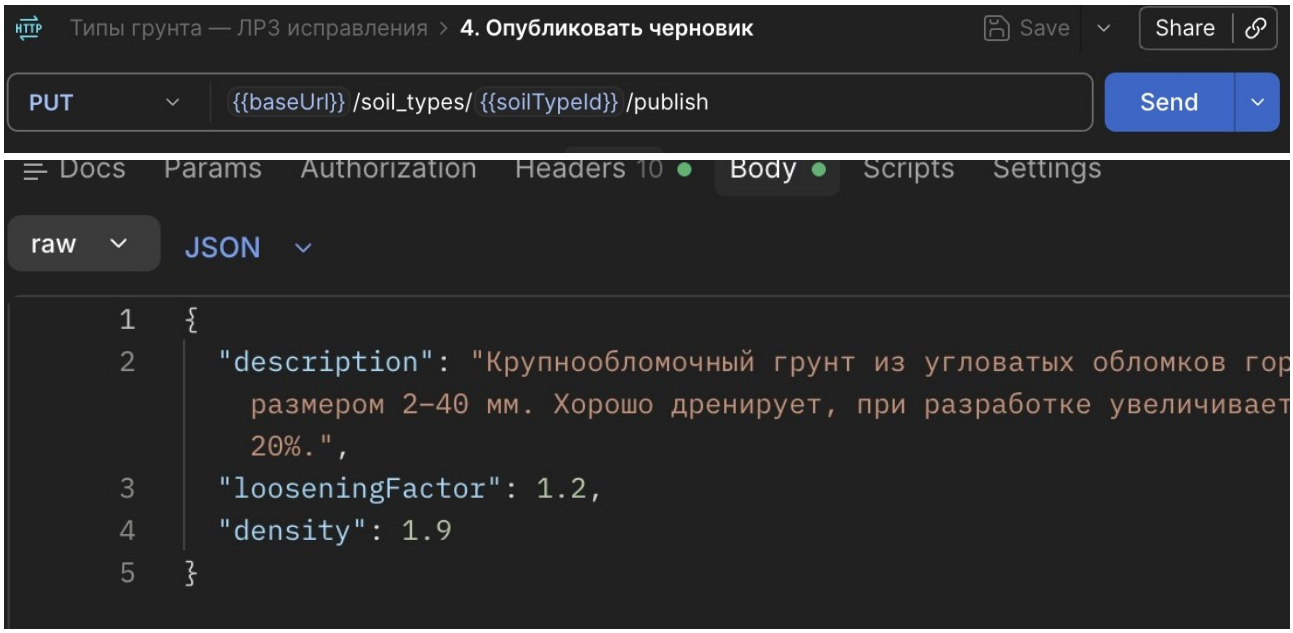
Скриншот 3 Черновик

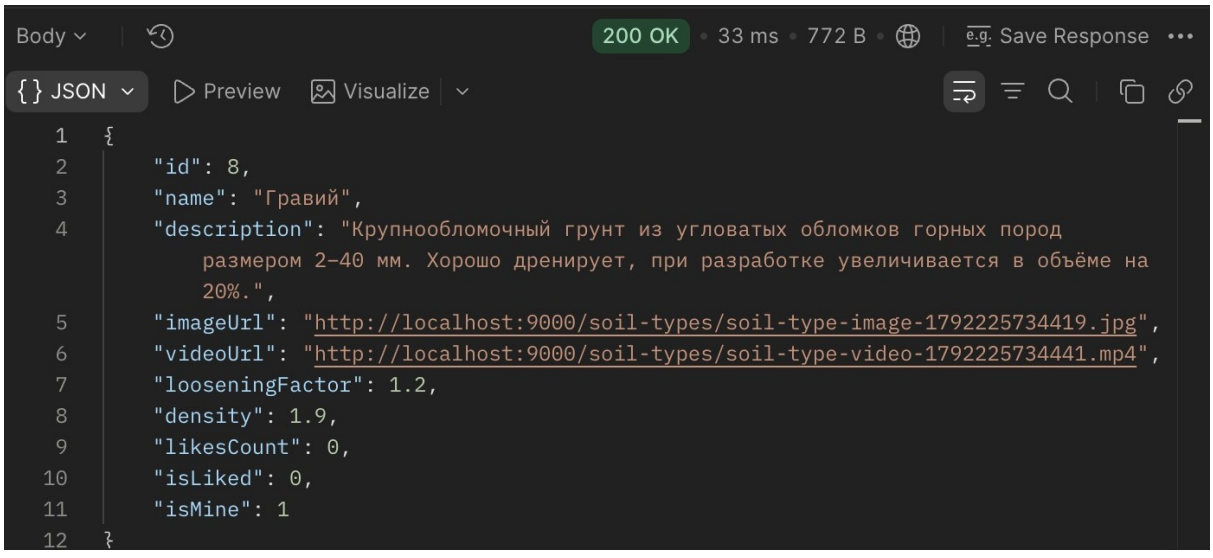
GET /api/soil_types/draft. Идентификатор не передаётся; сервер определяет пользователя. Незаполненные description, looseningFactor и density равны null.



Скриншот 4 Публикация

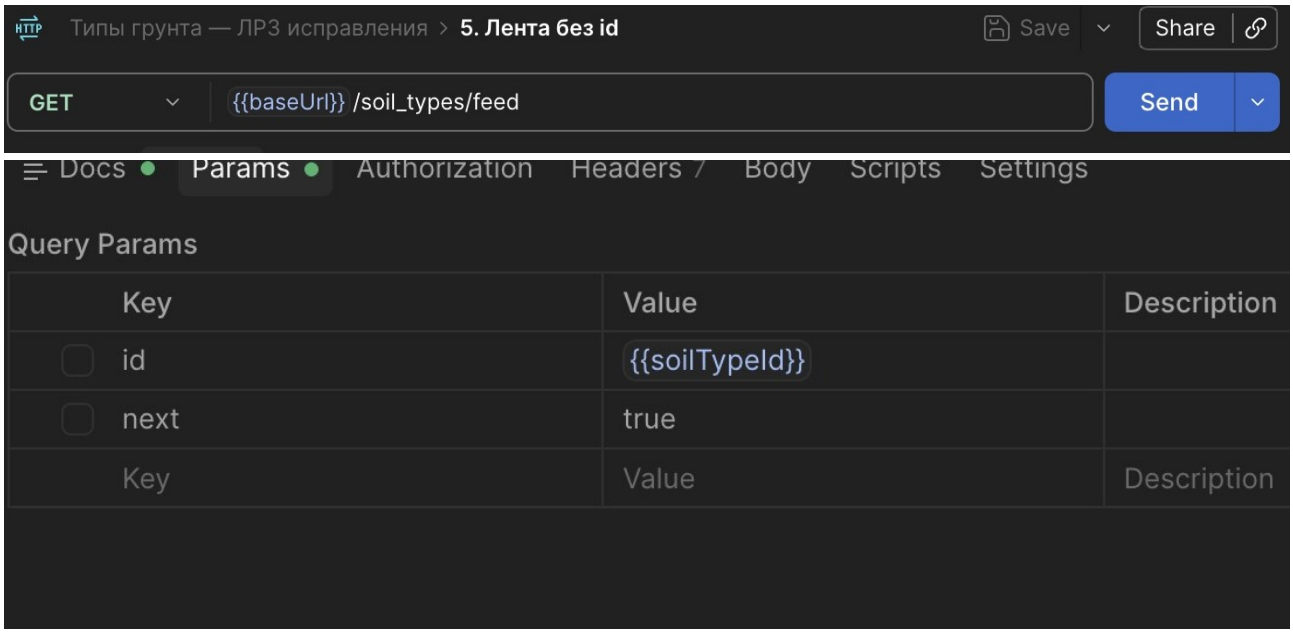
PUT /api/soil_types/8/publish. В JSON ровно три поля: description, looseningFactor, density. Статус и дата устанавливаются сервером.

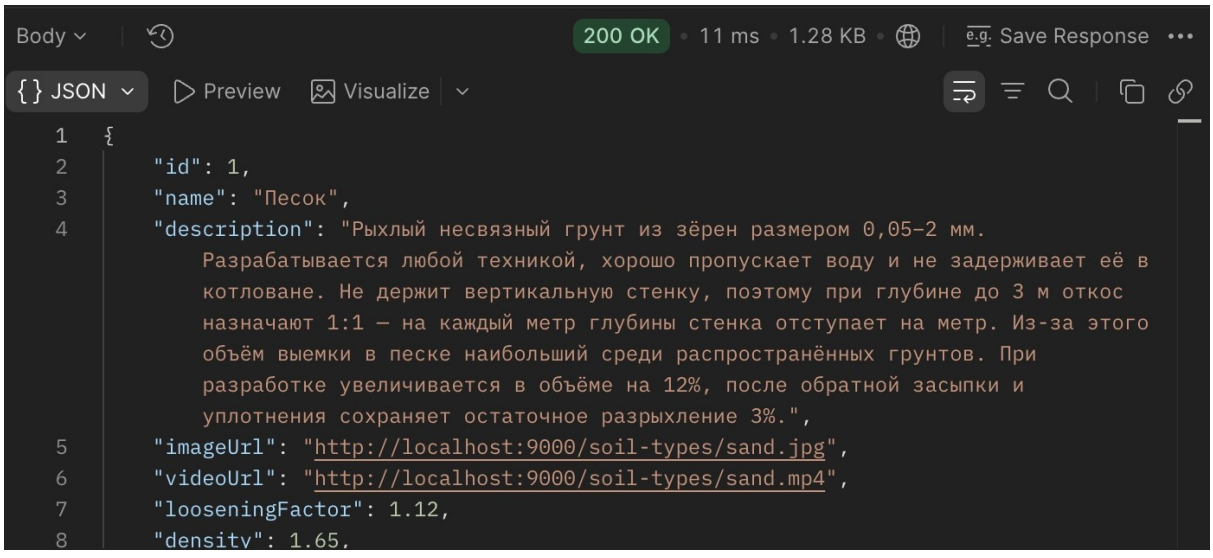




Скриншот 5 Лента без id

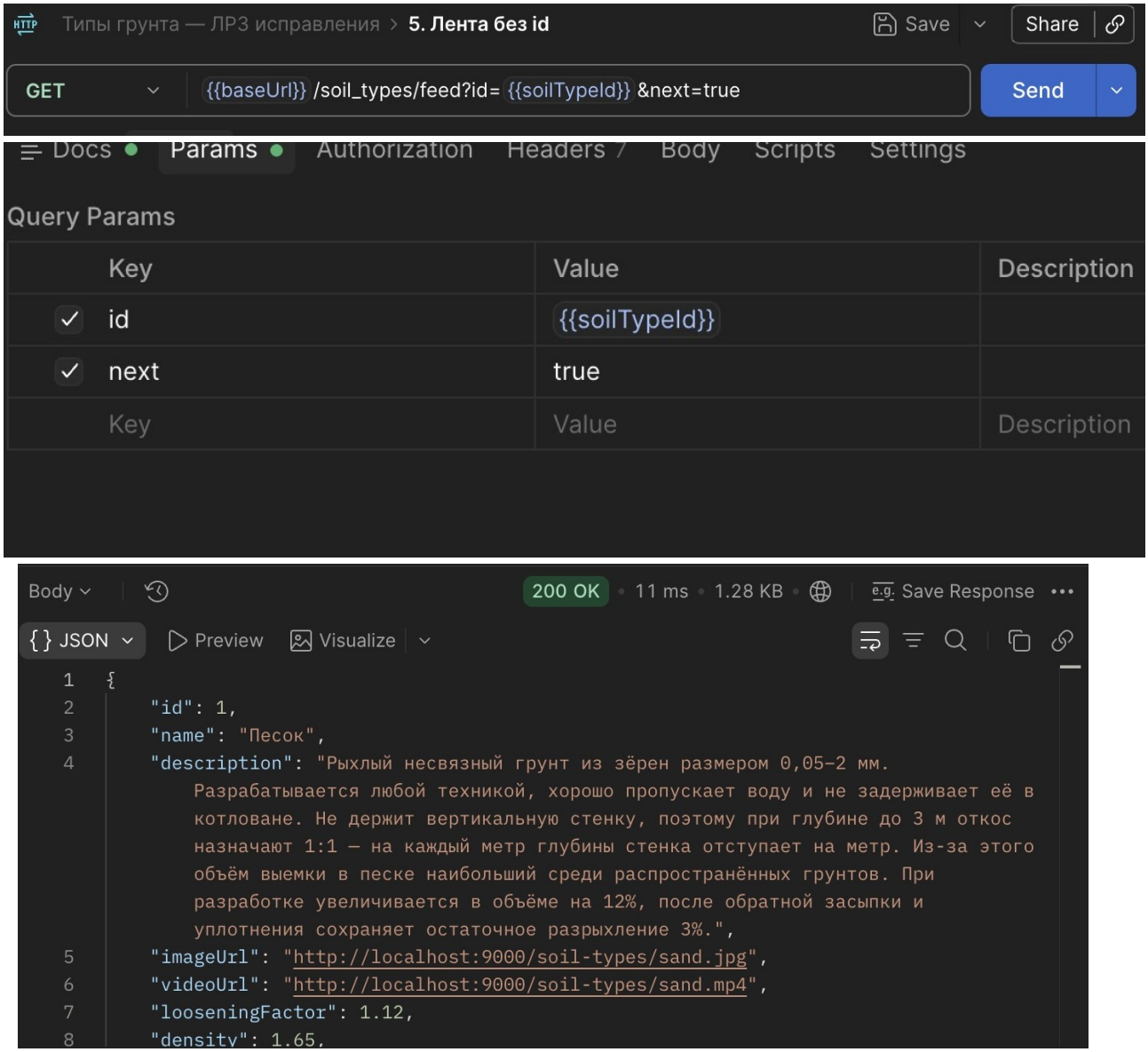
GET /api/soil_types/feed. Сервер возвращает первый опубликованный тип грунта. У песка isMine=0, поскольку его создал пользователь 2.





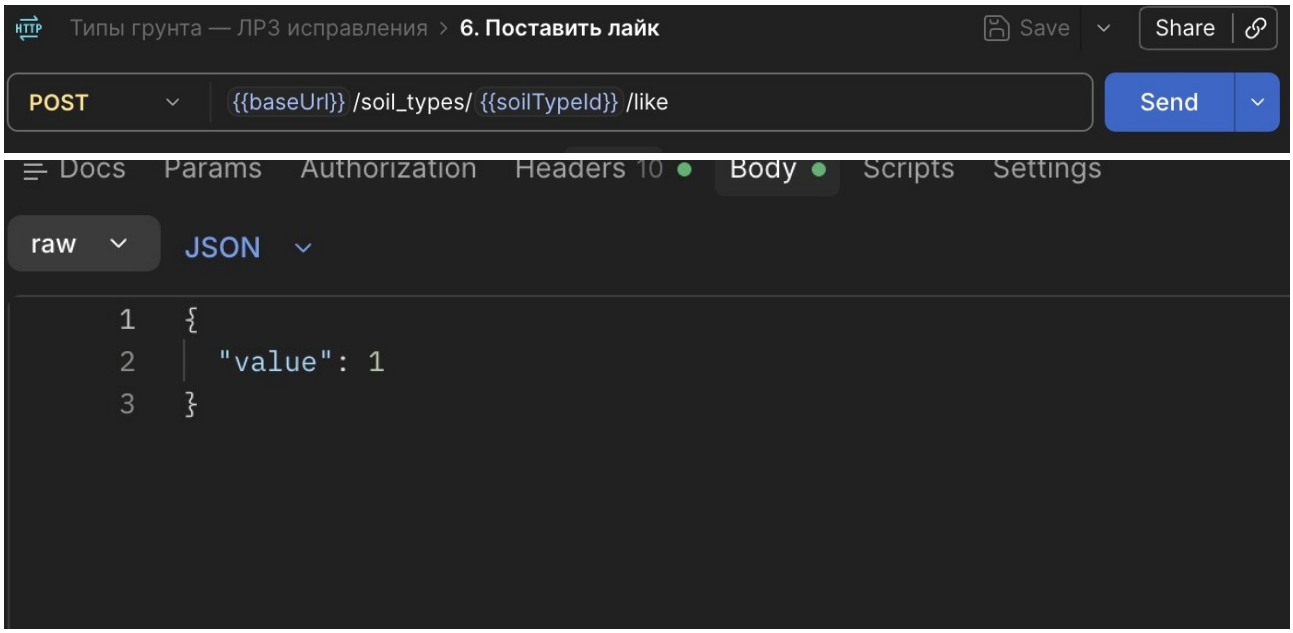
Скриншот 6 Следующий грунт

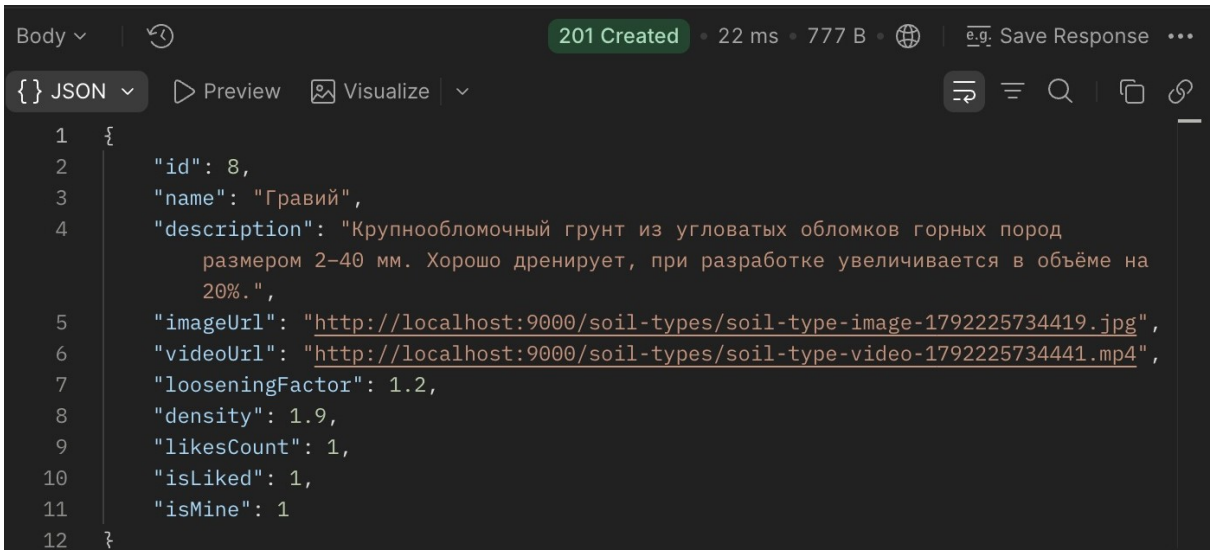
Тот же GET /api/soil_types/feed с id=8 и next=true. После последней карточки возвращается песок. Это второй вызов одного метода, а не отдельный метод API.



Скриншот 7 Лайк

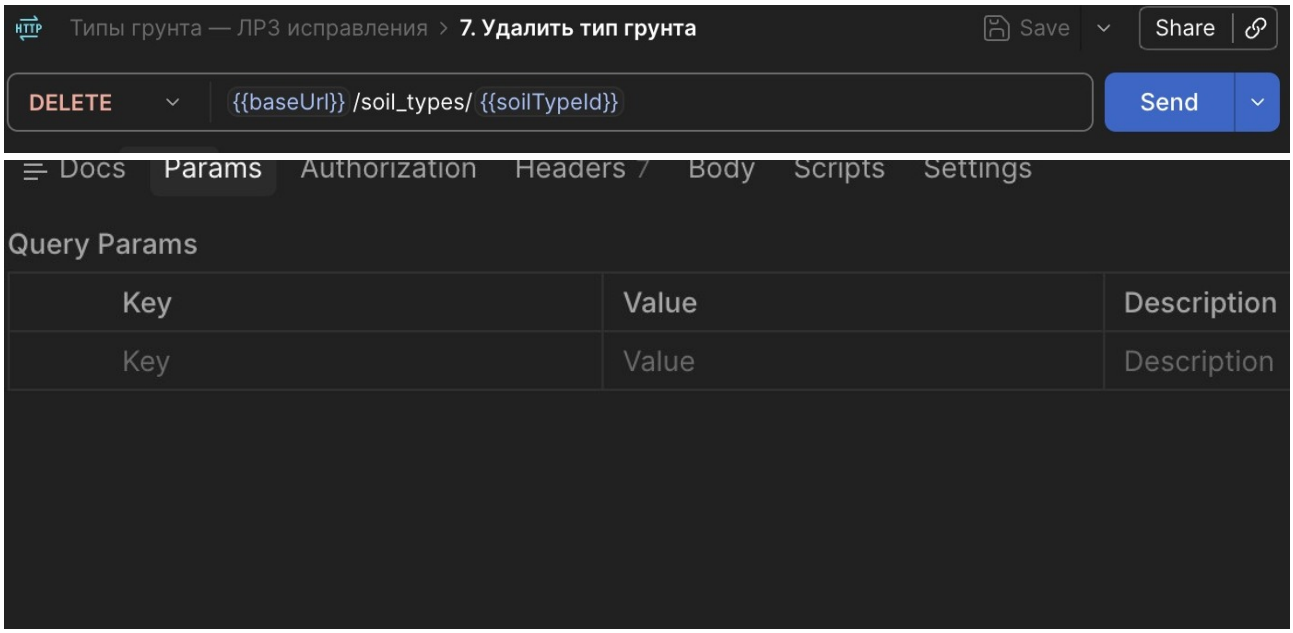
POST /api/soil_types/8/like. Вход value=1 ставит лайк, value=0 снимает. Пользователь определяется сервером.

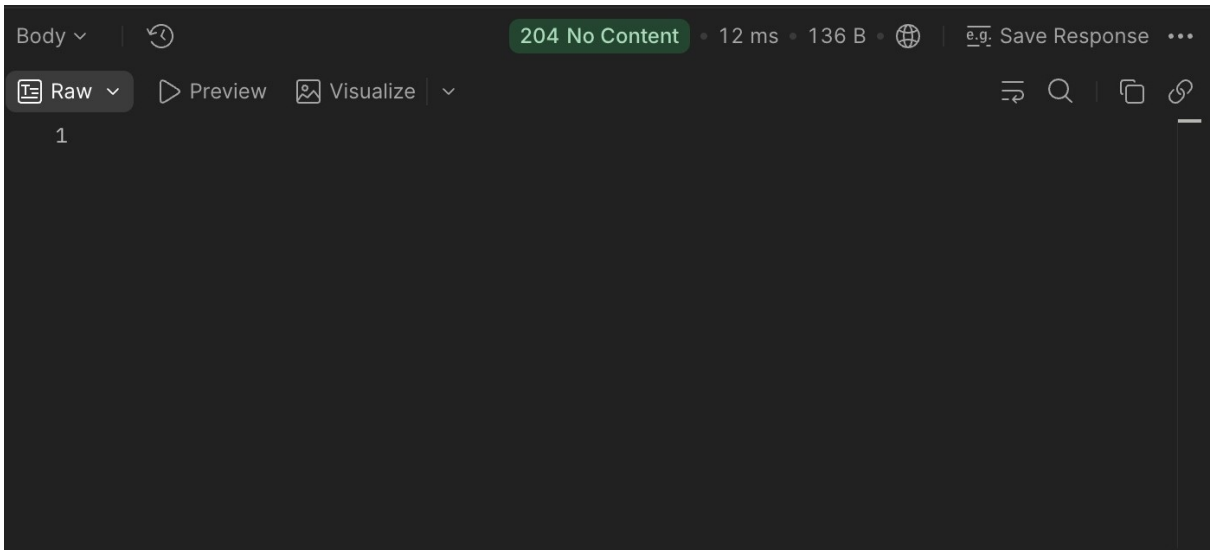




Скриншот 8 Удаление

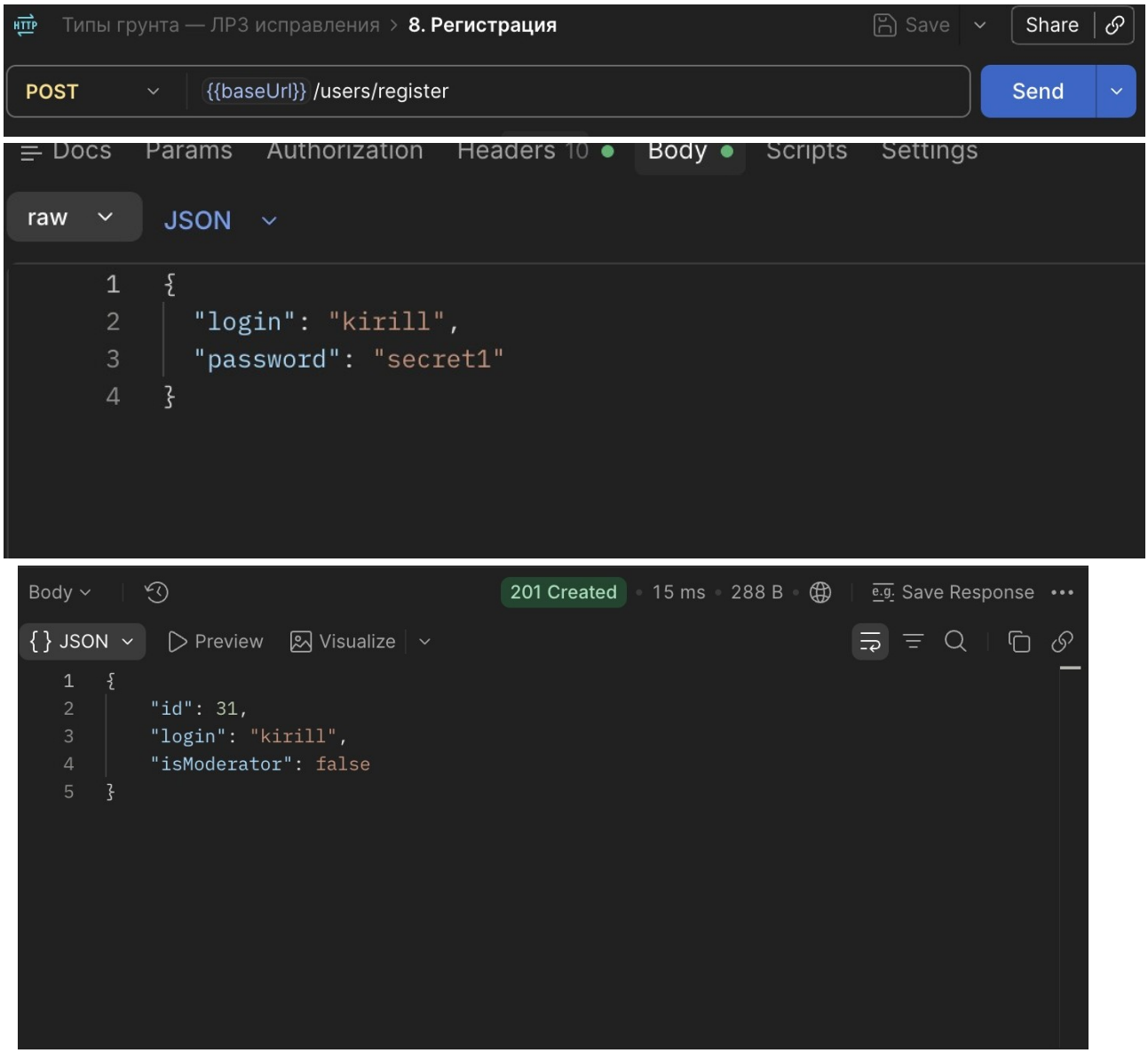
DELETE /api/soil_types/8. Ответ 204, строка остаётся со статусом deleted. Удалять можно только свои карточки.





Скриншот 9 Регистрация

POST /api/users/register. Вход login и password, ответ 201 без пароля.



Скриншот 11 Изменённые данные

Глина опубликована, Гравий после публикации удалён логически. creatorId песка и супеси равен 2.

PostgreSQL — 11-soil-types

```
SELECT id, name, status, "creatorId", "formedAt" FROM soil_types ORDER BY id;

 id |   name    |  status   | creatorId |          formedAt
----+-----------+-----------+-----------+----------------------------
  1 | Песок     | published |         2 | 2026-10-17 08:17:23.539394
  2 | Супесь    | published |         2 | 2026-10-17 08:17:23.539394
  3 | Суглинок  | published |         1 | 2026-10-17 08:17:23.539394
  4 | Галечник  | published |         1 | 2026-10-17 08:17:23.539394
  5 | Известняк | published |         1 | 2026-10-17 08:17:23.539394
  6 | Глина     | published |         1 | 2026-10-17 08:28:47.286234
  7 | Лёсс      | deleted   |         1 |
  8 | Гравий    | deleted   |         1 | 2026-10-17 08:31:09.847371
(8 rows)
```

Скриншот 12 Изменённые данные

URL файлов относятся к загрузке в показанном сценарии; лайк пользователя 1 хранится отдельной строкой.

PostgreSQL — 12-media-likes

```
SELECT id, "imageUrl", "videoUrl" FROM soil_types WHERE id = 8; SELECT * FROM likes WHERE
"soilTypeId" = 8;

 id | userId | soilTypeId
----+--------+------------
 51 |      1 |          8
(1 row)
```

Скриншот 13 Изменённые данные

После регистрации в таблице появился пользователь kirill. Пароль не включён в SELECT.

PostgreSQL — 13-users

```
SELECT id, login, "isModerator" FROM users WHERE id > 30;

 id | login  | isModerator
----+--------+-------------
 31 | kirill | f
(1 row)
```

Скриншот 14 Модель типа грунта

TypeORM описывает столбцы и связи. description, looseningFactor, density и formedAt допускают NULL. Поля likesCount и isLiked вычисляются запросом и не являются колонками.

src/entities/soil-type.entity.ts

```
import { Column, Entity, JoinColumn, ManyToOne, OneToMany, PrimaryGeneratedColumn } from
'typeorm';
import { Like } from './like.entity';
import { User } from './user.entity';

export type SoilTypeStatus = 'draft' | 'published' | 'deleted';

@Entity('soil_types')
export class SoilType {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ type: 'varchar', length: 100 })
  name: string;

  @Column({ type: 'varchar', length: 1000, nullable: true })
  description: string | null;

  @Column({ type: 'varchar', length: 20, default: 'draft' })
  status: SoilTypeStatus;

  @Column({ type: 'varchar', length: 255, default: '' })
  imageUrl: string;

  @Column({ type: 'varchar', length: 255, default: '' })
  videoUrl: string;

  @Column({ type: 'numeric', precision: 4, scale: 2, nullable: true })
  looseningFactor: number;

  @Column({ type: 'numeric', precision: 4, scale: 2, nullable: true })
  density: number;

  @Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })
  createdAt: Date;

  @Column({ type: 'timestamp', nullable: true })
  formedAt: Date | null;

  @Column({ type: 'int' })
  creatorId: number;

  @ManyToOne(() => User, { onDelete: 'NO ACTION' })
  @JoinColumn({ name: 'creatorId' })
  creator: User;

  @OneToMany(() => Like, (like) => like.soilType)
  likes: Like[];

  likesCount: number;

  isLiked: number;
}
```

Скриншот 14 Пользователи

Модель User содержит логин, пароль и признак модератора; выходной DTO не отдаёт пароль.

src/entities/user.entity.ts

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ type: 'varchar', length: 50 })
  login: string;

  @Column({ type: 'varchar', length: 100 })
  password: string;

  @Column({ type: 'boolean', default: false })
  isModerator: boolean;
}
```

Скриншот 14 Лайки

Like связывает пользователя с типом грунта двумя внешними ключами, каскадное удаление запрещено.

src/entities/like.entity.ts

```
import { Column, Entity, JoinColumn, ManyToOne, PrimaryGeneratedColumn } from 'typeorm';
import { SoilType } from './soil-type.entity';
import { User } from './user.entity';

@Entity('likes')
export class Like {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ type: 'int' })
  userId: number;

  @Column({ type: 'int' })
  soilTypeId: number;

  @ManyToOne(() => User, { onDelete: 'NO ACTION' })
  @JoinColumn({ name: 'userId' })
  user: User;

  @ManyToOne(() => SoilType, { onDelete: 'NO ACTION' })
  @JoinColumn({ name: 'soilTypeId' })
  soilType: SoilType;
}
```

Скриншот 15 DTO ответа

SoilTypeResponseDto исключает статус, даты и создателя, преобразует NUMERIC в number, рассчитывает isMine.

src/soil_types/dto/soil-type-response.dto.ts

```
import { SoilType } from '../../entities/soil-type.entity';

export class SoilTypeResponseDto {
  id: number;
  name: string;
  description: string | null;
  imageUrl: string | null;
  videoUrl: string | null;
  looseningFactor: number | null;
  density: number | null;
  likesCount: number;
  isLiked: number;
  isMine: number;

  static fromEntity(soilType: SoilType, currentUserId: number): SoilTypeResponseDto {
    return {
      id: soilType.id,
      name: soilType.name,
      description: soilType.description,
      imageUrl: soilType.imageUrl || null,
      videoUrl: soilType.videoUrl || null,
      looseningFactor: soilType.looseningFactor === null ? null :
Number(soilType.looseningFactor),
      density: soilType.density === null ? null : Number(soilType.density),
      likesCount: soilType.likesCount,
      isLiked: soilType.isLiked,
      isMine: soilType.creatorId === currentUserId ? 1 : 0,
    };
  }
}
```

Скриншот 15 DTO публикации

Три пользовательских поля. Декораторы проверяют обязательность, типы и диапазоны; глобальный ValidationPipe отклоняет неизвестные поля.

src/soil_types/dto/publish-soil-type.dto.ts

```
import { Type } from 'class-transformer';
import { IsNotEmpty, IsNumber, IsString, Max, MaxLength, Min } from 'class-validator';

export class PublishSoilTypeDto {
  @IsString()
  @IsNotEmpty()
  @MaxLength(1000)
  description: string;

  @Type(() => Number)
  @IsNumber()
  @Min(1)
  @Max(2)
  looseningFactor: number;

  @Type(() => Number)
  @IsNumber()
  @Min(0.5)
  @Max(3)
  density: number;
}
```

Скриншот 16 Текущий пользователь

До ЛР4 userId=1 выдаётся одной функцией. Она используется в запросах лайков, поиске и создании черновика, проверке владельца и DTO ответа.

src/current-user.ts

```
const CURRENT_USER_ID = 1;

export function getCurrentUserId(): number {
  return CURRENT_USER_ID;
}
```

Скриншот 17 README

README перечисляет все десять методов и все поля таблиц. POST входа и выхода в ЛР3 являются заглушками: вход возвращает текущего пользователя, выход — 204 без тела. Полноценная сессия реализуется в следующей работе.

№	Метод и адрес	Вход
1	GET /api/soil_types	maxLoosening?
2	GET /api/soil_types/feed	id?, next?
3	GET /api/soil_types/draft	нет
4	POST /api/soil_types	name, image?, video?
5	PUT /api/soil_types/{id}/publish	id; description, looseningFactor, density
6	DELETE /api/soil_types/{id}	id
7	POST /api/soil_types/{id}/like	id; value=0/1
8	POST /api/users/register	login, password
9	POST /api/users/login	нет, заглушка
10	POST /api/users/logout	нет, заглушка

soil_types

Поле	Тип	NULL	Назначение
id	SERIAL	нет	PK, генерируется БД
name	VARCHAR(100)	нет	Название типа грунта
description	VARCHAR(1000)	да	Описание; заполняется при публикации
status	VARCHAR(20)	нет	draft / published / deleted; сервер
imageUrl	VARCHAR(255)	нет	URL фото MinIO; пустая строка — заглушка
videoUrl	VARCHAR(255)	нет	URL видео MinIO; пустая строка — заглушка
looseningFactor	NUMERIC(4,2)	да	Коэффициент разрыхления, при публикации 1 –2
density	NUMERIC(4,2)	да	Плотность т/м³, при публикации 0,5–3
createdAt	TIMESTAMP	нет	БД: момент создания
formedAt	TIMESTAMP	да	БД: момент публикации
creatorId	INTEGER	нет	FK → users.id, сервер

users

Поле	Тип	NULL	Назначение
id	SERIAL	нет	PK
login	VARCHAR(50)	нет	Логин; уникальный в ЛР4
password	VARCHAR(100)	нет	В ЛР4 хеш bcrypt; в ответы не входит
isModerator	BOOLEAN	нет	Признак модератора; по умолчанию false

likes

Поле	Тип	NULL	Назначение
id	SERIAL	нет	PK
userId	INTEGER	нет	FK → users.id
soilTypeId	INTEGER	нет	FK → soil_types.id